

Supplementary Materials for: Similarity Network Fusion-Guided Deep Graph Learning for Single-Cell Multimodal Integration

Hongyu Duan, Yang Wang, Qianwen Chen, Xiaoxia Zhen, Jianxiu Jin, Huiling Liu, and Li C. Xia

S1. OVERVIEW OF THE SUPPLEMENTARY MATERIALS

This supplementary file provides additional implementation details, baseline settings, pseudotime analysis procedures, evaluation definitions, and computational complexity analysis for sxSNF. To avoid redundancy with the main manuscript, we do not repeat the main benchmark tables, main ablation table, or primary biological validation figures. Instead, the supplementary materials focus on information needed to reproduce and interpret the experiments.

All tables in this file are placed using non-floating placement ([H]) to avoid appearing before the supplementary title or being separated from their corresponding explanatory text.

S2. DETAILED SXSNF IMPLEMENTATION

A. Input preprocessing

For scRNA-seq data, genes detected in fewer than 10 cells were removed. Raw count matrices were library-size normalized, log-transformed, and used to select 2,000 highly variable genes. The selected expression matrix was scaled and projected into a low-dimensional principal component analysis (PCA) space using Scanpy [1]. The resulting PCA representation was used for RNA-specific cell-cell graph construction.

For scATAC-seq data, peaks detected in fewer than 100 cells were removed. The binary peak matrix was transformed using a TF-IDF-like weighting scheme and then projected into a low-dimensional latent semantic indexing (LSI) representation using singular value decomposition (SVD). The LSI representation was used for ATAC-specific cell-cell graph construction. For biological interpretation on the Chen-2019 dataset, ATAC-derived gene activity was computed by aggregating accessibility signals around gene bodies and proximal regulatory regions, followed by normalization before group-level visualization.

B. Modality-specific graph construction and SNF fusion

For each modality, pairwise cell similarities were computed from the low-dimensional representation and then converted into a sparse k -nearest-neighbor graph. Cosine distance was used as the default distance metric because it was more stable for sparse, high-dimensional single-cell representations. The local scaling parameter μ was set to 0.5 unless otherwise specified. Each modality-specific graph was normalized into a state matrix $P^{(v)}$ and a sparse local kernel matrix $S^{(v)}$. The iterative cross-diffusion update follows:

$$P_{t+1}^{(v)} = S^{(v)} \left(\frac{\sum_{u \neq v} P_t^{(u)}}{m-1} \right) \left(S^{(v)} \right)^T, \quad (\text{S1})$$

where m is the number of modalities and v indexes a modality. For the RNA+ATAC experiments in this study, $m = 2$. After t fusion iterations, the fused graph was obtained by averaging the final modality-specific state matrices. In the scalable implementation, only sparsified k NN affinity matrices are stored and updated, even though dense matrices are used in the mathematical notation for clarity.

C. Self-supervised graph learning

The input node-feature matrix for the graph neural network was constructed by concatenating the normalized low-dimensional RNA PCA and ATAC LSI representations. The fused graph was first sparsified by retaining the top- k neighbors for each cell and then symmetrized. The resulting sparse weighted graph was used for self-supervised masked-edge reconstruction.

During training, a subset of positive edges was randomly masked and used as reconstruction targets. Negative edges were sampled from node pairs that were not connected in the sparsified fused graph. Self-loops, duplicated pairs, and observed positive edges were excluded from the negative-edge pool. Unless otherwise stated, one negative edge was sampled for each masked positive edge. Negative edges were resampled during training to reduce sampling bias and to prevent the model from overfitting to a fixed set of negative pairs.

For an edge (u, v) , the predicted affinity was computed as $s_{uv} = \sigma(\mathbf{z}_u^T \mathbf{z}_v)$, where $\mathbf{z}_u$ and $\mathbf{z}_v$ are learned cell embeddings and $\sigma(\cdot)$ is the sigmoid function. The model was optimized using a weighted binary cross-entropy loss for masked-edge recovery and an additional observed-edge preservation term for non-masked positive edges.

TABLE S1: Summary of key sxSNF preprocessing, fusion, graph-learning, and clustering parameters.

Module	Parameter	Value / Description
Preprocessing	scRNA-seq minimum cells per gene	10
	scRNA-seq highly variable genes	2,000
	scATAC-seq minimum cells per peak	100
	scATAC-seq representation	TF-IDF-like weighting followed by SVD/LSI
SNF construction	Distance metric	Cosine distance by default
	Number of neighbors (k)	20
	Kernel scale (μ)	0.5
	Fusion iterations (t)	20
Self-supervised graph learning	Input node features	Concatenated normalized RNA PCA and ATAC LSI
	Encoder	Two-layer graph convolutional encoder
	Hidden dimension	128
	Final embedding dimension (d)	64
	Edge masking ratio (r_m)	0.2
	Negative sampling ratio	1:1 relative to masked positive edges
	Optimizer / learning rate	Adam / 1×10^{-3}
Training	Weight decay	1×10^{-4}
	Training epochs	200 with early stopping based on reconstruction loss
Leiden resolution	PBMC-10k	0.3
	Chen-2019	0.5
	SHARE-seq	0.4

S3. BASELINE METHODS AND COMPARATIVE SETTINGS

All baseline methods were evaluated on the same filtered cells and reference annotations as sxSNF. When a method produced a low-dimensional embedding rather than cluster labels, we applied the same downstream Leiden clustering and metric-calculation pipeline used for sxSNF. Default parameters from the original implementations or tutorials were used unless dataset-specific preprocessing was required. This strategy was used to reduce post-processing differences and to make the comparison focus on the learned representations.

TABLE S2: Summary of baseline methods and implementation settings used for comparison.

Method	Integration strategy	Implementation details
Seurat WNN [2]	Weighted nearest-neighbor fusion of modality-specific neighborhood structures.	RNA was processed with PCA and ATAC with LSI following the standard WNN workflow. The learned WNN graph or embedding was evaluated using the common Leiden and metric pipeline.
MultiVI [3]	Deep generative model for joint representation learning from multiome data.	The model was run using the matched RNA+ATAC input format and recommended defaults. The latent representation was extracted and clustered with the common downstream protocol.
GLUE [4]	Graph-linked unified embedding using feature guidance and modality-specific encoders.	RNA and ATAC feature representations were prepared according to the GLUE workflow. The learned cell embedding was evaluated under the same clustering and metric scripts.
scJoint [5]	Neural-network-based joint embedding with cross-modality alignment.	Default training settings were used where applicable. The resulting embedding was clustered using the shared Leiden protocol.
SIMBA [6]	Graph embedding framework that represents cells and features in a shared graph.	The standard graph construction and embedding workflow was used. Cell embeddings were extracted for downstream Leiden clustering.
scMIC [7]	Deep representation learning for single-cell multimodal integration.	The official or recommended parameter setting was used. Results were evaluated on the same filtered cell sets.
DCCA [8]	Correlation-based deep representation learning baseline.	RNA and ATAC low-dimensional representations were used as paired inputs. The learned representation was evaluated through the common clustering pipeline.
Guanlab-Dengkw [9]	Competitive multimodal integration baseline from recent comparative analyses.	The method was run using the same cell set and available default settings. Output embeddings or labels were evaluated with the shared metric scripts.
Spectral clustering [10]	Conventional graph-based clustering baseline.	A similarity graph was constructed from the integrated or concatenated representation and directly clustered using spectral clustering.

A. Fair-comparison considerations

For all methods, the same filtered cells and reference annotations were used. Random seeds were kept consistent across repeated runs whenever supported by the method implementation. For methods requiring modality-specific preprocessing, RNA PCA and ATAC LSI were generated from the same filtered matrices used for sxSNF. For methods with their own recommended preprocessing pipelines, we followed the corresponding tutorial or official documentation, while retaining the same cell set for final evaluation.

Leiden resolution was selected at the dataset level according to the expected annotation granularity and then fixed for all repeated runs within that dataset. The resolution was not tuned separately for each random seed. For embedding-based methods, this common downstream clustering protocol was used to avoid method-specific resolution optimization.

S4. PSEUDOTIME ANALYSIS OF OLIGODENDROCYTE-LINEAGE SUBGROUPS

A. Lineage-restricted cell selection

To evaluate whether sxSNF provides biological information beyond recovering coarse cell-type annotations, we performed a lineage-restricted analysis of oligodendrocyte-lineage cells in the Chen-2019 dataset. Cells annotated as oligodendrocyte precursor cells (OPC), intermediate oligodendrocytes, and mature oligodendrocytes were selected from the integrated sxSNF embedding. These cells were reclustered in the sxSNF latent space to identify refined subgroups.

B. Diffusion pseudotime procedure

Diffusion pseudotime (DPT) was used to order the refined oligodendrocyte-lineage subgroups along a maturation-related trajectory. DPT was computed on a nearest-neighbor graph constructed from the sxSNF embedding of lineage-restricted cells. The root cell was selected from the subgroup enriched for OPC-like or early-lineage markers, such as *Pdgfra*, *Cspg4*, or related precursor-state signatures, when available. If multiple candidate root cells showed comparable precursor-marker profiles, the root was selected from the centroid-nearest cell of the OPC-enriched subgroup to improve reproducibility.

The DPT workflow consisted of the following steps:

- 1) Select oligodendrocyte-lineage cells from the Chen-2019 dataset based on the original annotations.
- 2) Construct a nearest-neighbor graph using the sxSNF latent embedding restricted to the selected cells.

- 3) Identify an OPC-like or early-lineage root cell based on precursor-enriched marker expression and subgroup composition.
- 4) Compute diffusion components and DPT values using the neighborhood graph.
- 5) Visualize DPT values on the lineage-restricted UMAP embedding and compare the ordering with subgroup marker programs.

DPT values were interpreted as relative maturation-related ordering rather than direct temporal measurements. Therefore, the pseudotime analysis was used as supportive evidence for the organization of refined subgroups and was interpreted together with marker-gene expression, subgroup composition, and ATAC-derived gene activity.

TABLE S3: Key settings and interpretation criteria for the DPT pseudotime analysis.

Item	Description
Input cells	Oligodendrocyte-lineage cells from the Chen-2019 dataset, including OPC-like, intermediate, and mature oligodendrocyte populations.
Input representation	sxSNF latent embedding restricted to the selected lineage cells.
Graph construction	Nearest-neighbor graph constructed from the lineage-restricted sxSNF embedding.
Root selection	OPC-like subgroup or centroid-nearest cell from the subgroup enriched for early-lineage markers.
Output	Relative DPT ordering visualized on the lineage-restricted UMAP embedding.
Interpretation	Supportive maturation-related ordering; not interpreted as direct experimental time.

C. Marker programs used to support subgroup ordering

The pseudotime ordering was interpreted together with oligodendrocyte-lineage marker programs. Precursor-like or early-lineage states were evaluated using markers such as *Pdgfra* and *Cspg4*. Mature myelinating states were supported by canonical oligodendrocyte markers including *Plp1*, *Mbp*, *Mag*, *Mog*, *Mobp*, and *Aspa*. Transitional subgroups were interpreted based on intermediate expression patterns and their position between OPC-like and mature myelinating groups in the sxSNF embedding and DPT ordering.

S5. DIFFERENTIAL EXPRESSION AND MARKER-GENE VALIDATION

Cluster identity validation in the Chen-2019 dataset was performed using marker-gene visualization and differential expression analysis. For each annotated cell population, differential expression was computed by comparing the target cluster against the remaining cells. Genes were ranked according to differential-expression statistics, and cluster-specific genes were visualized using volcano plots and marker heatmaps. These analyses were used to verify that the sxSNF embedding preserved biologically meaningful transcriptional programs.

For marker visualization, both broad cell-class markers and more specific subtype markers were inspected. Astrocytes were assessed using glial markers such as *Gpc5*. Mature oligodendrocytes were assessed using myelin-related genes such as *Plp1* and *Mbp*. Excitatory neuron subtypes were evaluated using layer-associated markers such as *Rorb* and *Il1rapl2*. Inhibitory interneuron groups were evaluated using interneuron-related markers such as *Adarb2*. These marker analyses were used as biological support for the cell-type identities recovered in the integrated representation.

S6. INTEGRATION-QUALITY METRICS

In addition to ARI, NMI, and AMI, integration quality was assessed using neighborhood-based batch mixing, modality mixing, and alignment score. These metrics were computed on the k -nearest-neighbor graph of the learned embedding.

For a cell i , let $N_k(i)$ denote the set of its k nearest neighbors in the embedding space. Batch mixing was computed as the observed fraction of neighbors from batches different from the query cell, normalized by the expected fraction under random mixing:

$$\text{BatchMix}(i) = \frac{\frac{1}{k} \sum_{j \in N_k(i)} \mathbb{I}[b_j \neq b_i]}{1 - \sum_b p_b^2}, \quad (\text{S2})$$

where b_i is the batch label of cell i and p_b is the global proportion of batch b . The reported score was averaged across cells.

Modality mixing was computed analogously by replacing batch labels with modality labels:

$$\text{ModMix}(i) = \frac{\frac{1}{k} \sum_{j \in N_k(i)} \mathbb{I}[m_j \neq m_i]}{1 - \sum_m p_m^2}, \quad (\text{S3})$$

where m_i is the modality label and p_m is the global proportion of modality m .

For alignment score, we evaluated whether cross-modality neighbors preserved biological cell-type identity:

$$\text{Align}(i) = \frac{\frac{1}{|C_i|} \sum_{j \in C_i} \mathbb{I}[y_j = y_i]}{\sum_y p_y^2}, \quad (\text{S4})$$

where $C_i = \{j \in N_k(i) : m_j \neq m_i\}$ is the set of cross-modality neighbors of cell i , y_i is the annotated cell type, and p_y is the global proportion of cell type y . Cells without cross-modality neighbors were excluded from the per-cell alignment average. Higher values indicate better local mixing or cross-modality biological alignment.

S7. COMPUTATIONAL COMPLEXITY AND SCALABILITY

This section corresponds to the computational complexity and scalability analysis moved from Section III-J of the main manuscript to the supplementary materials.

The computational cost of sxSNF arises from two major stages: (1) similarity network fusion and (2) self-supervised graph learning on the fused network. Below we summarize the key complexity characteristics of the framework.

A. SNF fusion stage

A major cost in sxSNF lies in constructing a k -nearest-neighbor graph for each modality. A brute-force KNN search requires computing all pairwise distances among n cells, resulting in a quadratic $O(n^2)$ time complexity, which quickly becomes prohibitive for large single-cell datasets.

To avoid this bottleneck, approximate nearest-neighbor indices such as HNSW [11] and FAISS [12] can be used. These structures perform hierarchical or graph-based search, reducing per-query complexity to approximately $O(\log n)$. As a consequence, constructing the full KNN graph scales near $O(n \log n)$, which is suitable for 10^5 – 10^6 cells. Table S4 summarizes the complexity differences between exact and ANN-based KNN graph construction.

TABLE S4: Comparison of exact KNN and ANN-based KNN construction.

Method	Time complexity	Memory	Practical scale
KNN (brute force)	$O(n^2)$	$O(n)$	$< 5 \times 10^4$ cells
HNSW (ANN)	$O(n \log n)$	$O(n)$	10^5 – 10^6 cells
FAISS (IVF/HNSW)	$O(n \log n)$	$O(n)$	10^5 – 10^7 cells

Once modality-specific KNN graphs are constructed, SNF performs iterative diffusion updates:

$$P^{(v)} \leftarrow S^{(v)} \cdot \left(\frac{1}{m-1} \sum_{u \neq v} P^{(u)} \right) \cdot (S^{(v)})^T, \quad (\text{S5})$$

where each update touches only the k nearest neighbors of each cell. Therefore, each iteration requires approximately $O(nk^2)$, and the full SNF procedure over t iterations and m modalities grows close to linearly with n when k and t are fixed. Although dense matrices are used in the mathematical notation for clarity, the implementation stores and updates sparsified k NN affinity matrices, avoiding dense $n \times n$ memory usage in the scalable setting.

B. Graph learning stage

After fusion, the self-supervised graph neural network operates on the fused sparse graph with $|E| \approx nk$ edges. Each GNN layer performs message passing with complexity approximately $O(|E|)$, and because the number of layers and embedding dimension are small, this stage also scales nearly linearly with the number of cells.

C. Overall scalability

Combining both stages, the total runtime of sxSNF increases as:

$$O(n \log n) + O(nk^2) + O(nk), \quad (\text{S6})$$

dominated by KNN construction and SNF diffusion for small k values, typically 10–30. Memory usage is also linear in n since all graphs remain sparse.

To empirically assess scalability, runtime can be measured across increasing subsets of PBMC-10k and SHARE-seq. SNF is expected to be the dominant cost, whereas the GNN contributes a smaller fraction of runtime. sxSNF scales reliably to 10^5 – 5×10^5 cells on a single GPU workstation. For million-cell datasets, the method can leverage approximate nearest-neighbor indices, graph partitioning, and GPU-accelerated sparse kernels to maintain efficiency, making sxSNF suitable for large-scale multimodal single-cell atlases.

S8. ADDITIONAL ROBUSTNESS ANALYSIS

A. Effects of distance measures

We further examined the robustness of sxSNF by evaluating the influence of key components in the fusion framework. We investigated the role of distance measures in constructing the initial modality-specific cell graphs. As summarized in Table S5, cosine distance consistently outperformed chi-square distance across both sxSNF and SNF+scMIC, yielding higher ARI, NMI, and AMI scores. This advantage is likely due to the high sparsity and scale variability of single-cell molecular matrices, where cosine distance better captures directional consistency in expression patterns rather than absolute magnitude. Notably, sxSNF

exhibited smaller performance drops than SNF+scMIC when switching from cosine to chi-square distance, suggesting that the fused representation learned by sxSNF is more robust to shifts in initial graph construction. These analyses indicate that cosine distance provides a more stable foundation for constructing cell–cell affinity graphs in multimodal fusion. Moreover, the relatively consistent performance of sxSNF across distance metrics highlights its resilience against variations in preprocessing and its suitability for diverse multimodal single-cell datasets.

TABLE S5: Effects of distance measures on clustering results in an auxiliary distance-sensitivity experiment. This analysis uses the same evaluation pipeline but is reported separately from the main PBMC-10k benchmark in the manuscript.

Methodologies	Composition	ARI	NMI	AMI
sxSNF	cosine distance	0.5719	0.7582	0.7351
	chi-square distance	0.5443	0.7179	0.7206
SNF+scMIC	cosine distance	0.3811	0.6047	0.5928
	chi-square distance	0.3024	0.5223	0.5176

S9. REPRODUCIBILITY NOTES

All reported clustering and integration metrics were computed from the same filtered cell sets used in the main experiments. Repeated runs used identical random seeds across methods whenever applicable. For methods that produced embeddings rather than cluster labels, clustering was performed using the same Leiden-based downstream pipeline. These settings were used to minimize variability introduced by different clustering or metric-calculation scripts.

The supplementary tables intentionally exclude the main benchmark results and main ablation results to avoid duplicating content already reported in the main manuscript. The focus of this file is on implementation details, additional methodological explanations, pseudotime analysis, baseline configuration, and scalability analysis.

REFERENCES

- [1] F. A. Wolf, P. Angerer, and F. J. Theis, “Scanpy: large-scale single-cell gene expression data analysis,” *Genome biology*, vol. 19, pp. 1–5, 2018.
- [2] Y. Hao, S. Hao, E. Andersen-Nissen, W. M. Mauck, S. Zheng, A. Butler, M. J. Lee, A. J. Wilk, C. Darby, M. Zager *et al.*, “Integrated analysis of multimodal single-cell data,” *Cell*, vol. 184, no. 13, pp. 3573–3587, 2021.
- [3] T. Ashuach, M. I. Gabitto, R. V. Koodli, G.-A. Saldi, M. I. Jordan, and N. Yosef, “Multivi: deep generative model for the integration of multimodal data,” *Nature Methods*, vol. 20, no. 8, pp. 1222–1231, 2023.
- [4] Z.-J. Cao and G. Gao, “Multi-omics single-cell data integration and regulatory inference with graph-linked embedding,” *Nature Biotechnology*, vol. 40, no. 10, pp. 1458–1466, 2022.
- [5] Y. Lin, T.-Y. Wu, S. Wan, J. Y. Yang, W. H. Wong, and Y. R. Wang, “scjoint integrates atlas-scale single-cell rna-seq and atac-seq data with transfer learning,” *Nature biotechnology*, vol. 40, no. 5, pp. 703–710, 2022.
- [6] H. Chen, J. Ryu, M. E. Vinyard, A. Lerer, and L. Pinello, “Simba: single-cell embedding along with features,” *Nature Methods*, pp. 1–11, 2023.
- [7] Y. Zhan, J. Liu, and L. Ou-Yang, “scmic: A deep multi-level information fusion framework for clustering single-cell multi-omics data,” *IEEE Journal of Biomedical and Health Informatics*, 2023.
- [8] G. F. Zebende, “Dcca cross-correlation coefficient: Quantifying level of cross-correlation,” *Physica A: Statistical Mechanics and its Applications*, vol. 390, no. 4, pp. 614–618, 2011.
- [9] P. Rautenstrauch and U. Ohler, “Liam tackles complex multimodal single-cell data integration challenges,” *Nucleic Acids Research*, p. gkae409, 2024.
- [10] U. Von Luxburg, “A tutorial on spectral clustering,” *Statistics and computing*, vol. 17, pp. 395–416, 2007.
- [11] Y. A. Malkov and D. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 8, pp. 1872–1886, 2018.
- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” in *IEEE Transactions on Big Data*. IEEE, 2019.